08. Pytorch Wide and Deep.ipynb

Demo

Variable	description
age:	continuous
workclass:	Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, ...
fnlwgt:	continuous.
education:	Bachelors, Some-college, 11th, HS-grad, Prof-school, ...
education-num:	continuous.
marital-status:	Married-civ-spouse, Divorced, Never-married, Separated, Widowed, ...
occupation:	Tech-support, Craft-repair, Other-service, ...
relationship:	Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried.
race:	White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.
sex:	Female, Male.
capital-gain:	continuous.
capital-loss:	continuous.
hours-per-week:	continuous.
native-country:	United-States, Cambodia, England, ...
income (target):	<50k, >=50k



```
Test Labels
<=50K    11360
>50K      3700

Training Labels
<=50K    22654
>50K      7508

workclass 7 unique values.
education 16 unique values.
marital_status 7 unique values.
occupation 14 unique values.
relationship 6 unique values.
race 5 unique values.
sex 2 unique values.
country 41 unique values.
```

Creating Crossed Embeddings …

Wide and Deep, Setup

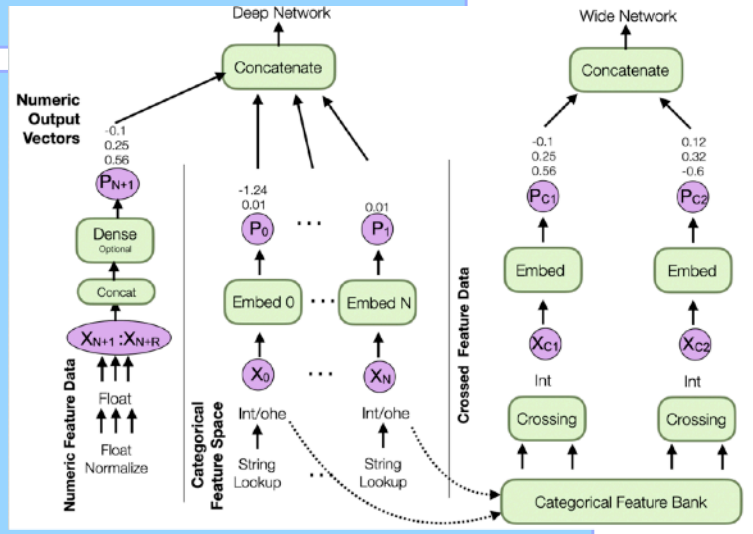
Demo

```
crossed_columns = []
crossing_sizes = {}
for feat1, feat2 in [('education','marital_status'), ('education','occupation') ]:
    col = f'{feat1}_x_{feat2}'
    df_train[col] = df_train[feat1].astype(str) + "_" + df_train[feat2].astype(str)

    crossed_columns.append(col)

    tmp = LabelEncoder()
    df_train[col+'_int'] = LabelEncoder().fit_transform(df_train[col])
    N = ... total classes ...
    emb_dim = int(np.sqrt(N + 1))
    crossing_sizes[col+'_int'] = (N, emb_dim)
```

feat1 (str)	feat2 (str)	crossing
Bachelors	married_af	Bachelors_married_af
Some HS	single	Some HS_single




```
class AdultCrossedDataset(Dataset):
    def __init__(self, df):
        # separate the categorical and feature crossings
        cat_cols = [x+'_int' for x in categorical_headers]
        crs_cols = [x+'_int' for x in crossed_columns]
        self.X_num = torch.tensor(df[numeric_headers], ...)
        self.X_cat = torch.tensor(df[cat_cols], ...)
        self.X_crs = torch.tensor(df[crs_cols], ...)
        self.y = torch.tensor(df.income, ...)

    def __getitem__(self, idx):
        return self.X_num[idx],   self.X_cat[idx],   self.X_crs[idx],   self.y[idx]

train_loader = DataLoader(AdultCrossedDataset(df_train), batch_size=64, shuffle=True)
```

Wide and Deep, Model



Demo

```
class IncomeWideDeepClassifier(nn.Module):
    def __init__(self, embedding, crossing, num_numerical_cols):
        ... same as code before ... and then ...
        self.embeddings = nn.ModuleList(tmp)
        self.deep_net   = nn.Sequential(… same as previous …)

        # feature crossing
        tmp = [],  total_crs_dim = 0
        for key, (vocab_size, embed_size) in crossing.items():
            tmp.append( nn.Embedding(vocab_size, embed_size) )
            total_crs_dim += embed_size

        self.crossings = nn.ModuleList(tmp)

        self.wide_net = nn.Sequential( nn.Linear(total_crs_dim, 8) )
        self.classify_net = nn.Sequential( nn.Linear(16, 1),
                                           nn.Sigmoid()     )

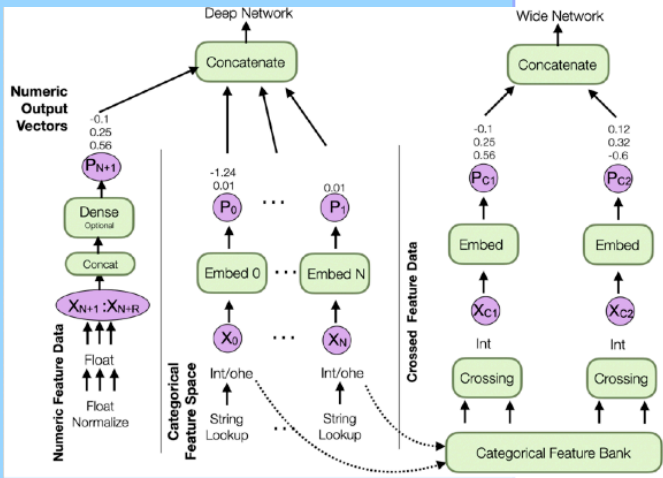
    def forward(self, x_num, x_cat, x_crs):
        emb_outputs = [emb(x_cat[:, i]) for i, emb in enumerate(self.embeddings)]

        x_emb = torch.cat(emb_outputs, dim=1)
        x = torch.cat([x_num, x_emb], dim=1)
        dx = self.deep_net(x)

        crs_outputs = [emb(x_crs[:, i]) for i, emb in enumerate(self.crossings)]
        x_emb_crs = torch.cat(crs_outputs, dim=1)
        cx = self.wide_net(x_emb_crs)

        x = torch.cat([dx, cx], dim=1)
        return self.classify_net(x)

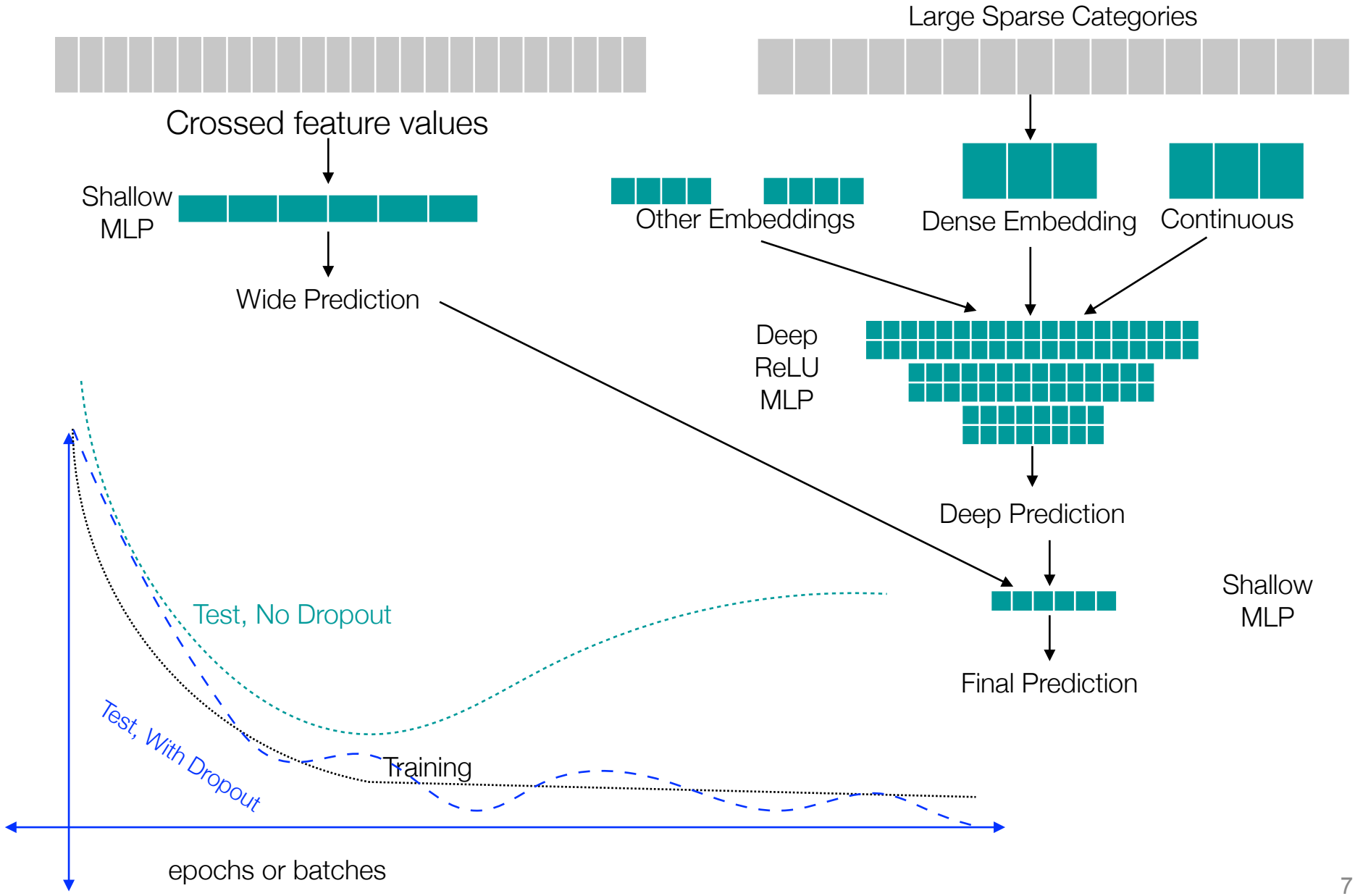

model = IncomeWideDeepClassifier(embedding_sizes, crossing_sizes, 4)
```



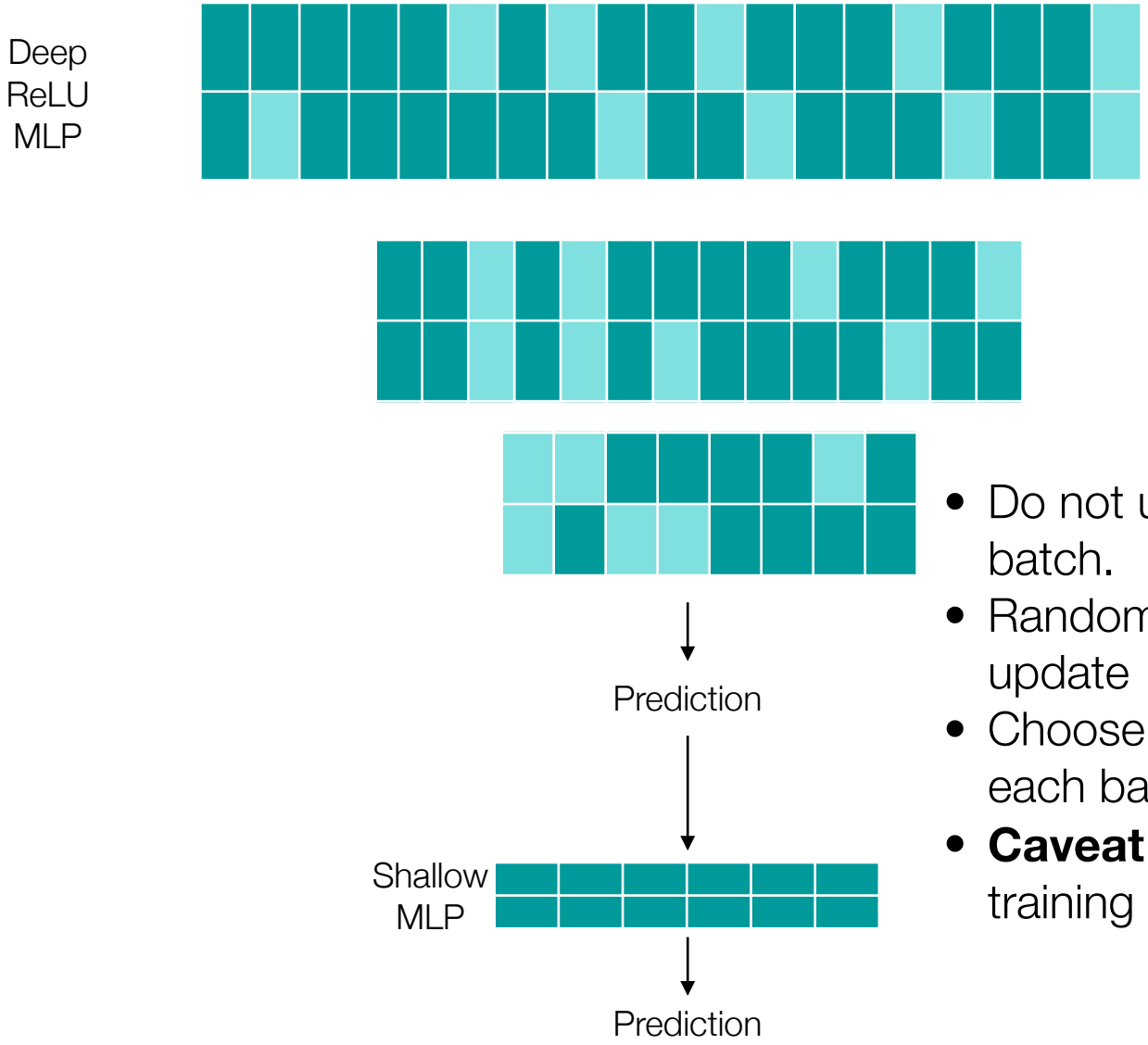
Best Results:

Numeric Only: 78.94%
Embedded: 84.81%
Crossed: 85.09%

Dropout: Helping prevent over fitting…



Dropout: Helping prevent over fitting…

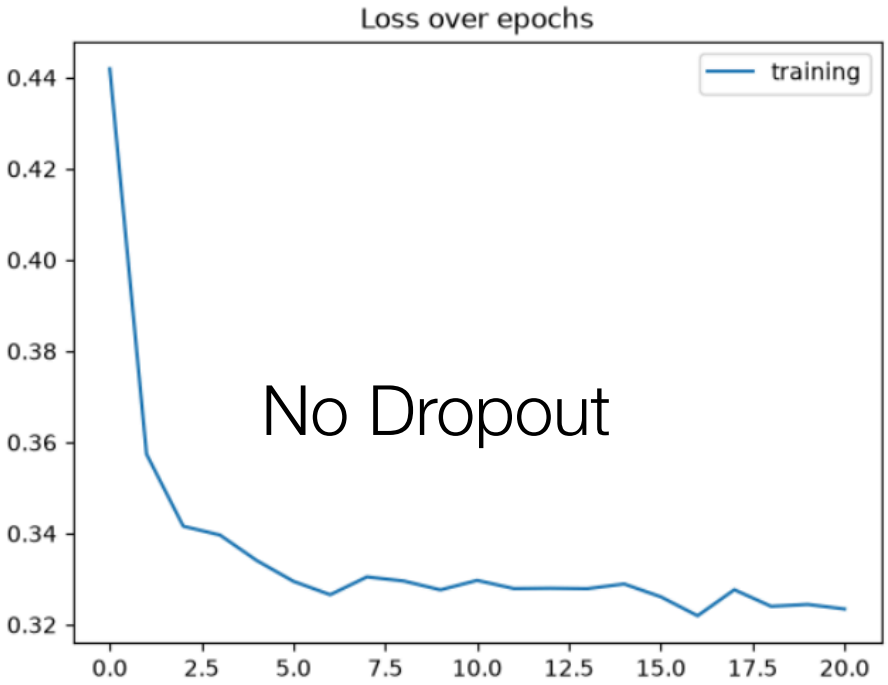


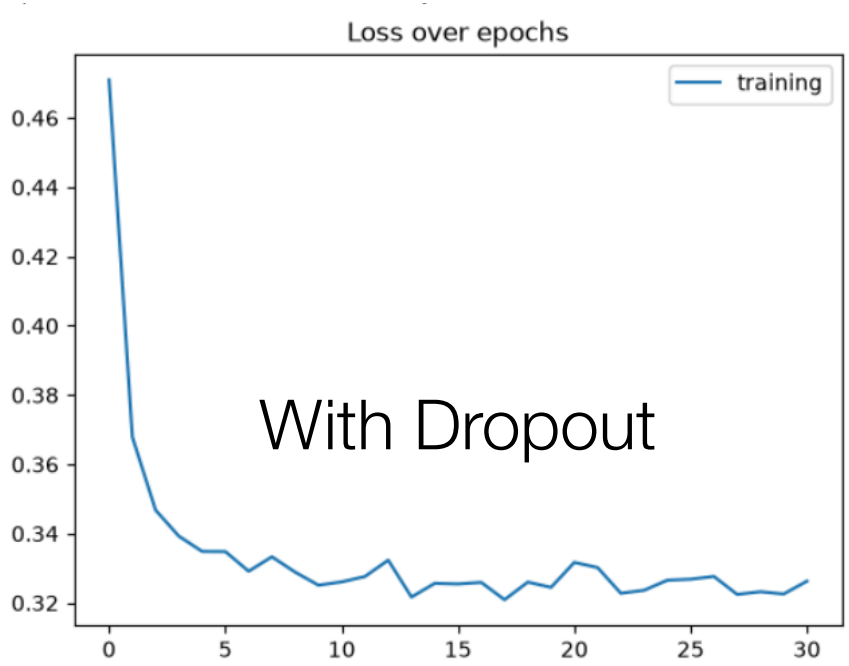
- Do not update all weights for each batch.
- Randomly choose which weights to update
- Choose different set of weights for each batch (random percentage)
- **Caveat**: does extend required training time for convergence

08. Pytorch Wide and Deep.ipynb



Demo





Best Results:

Numeric Only:	78.94%
Embedded:	84.81%
Crossed:	85.09%
Dropout:	85.25%

```
self.deep_net = nn.Sequential(
            nn.Linear(total_input_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, 16),
            nn.Dropout(p=0.1),
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.Dropout(p=0.25),
        )
```

For training, model must be in `model.train()` mode, then must use `model.eval()` for testing

Model convergence



loss (maybe log scaled)

epochs or batches

```
import wandb

wandb.init( …
   config={
   “learning_rate": 0.001,
   "architecture": "MLP",
   "dataset": “my_data",
   "epochs": 10, } )

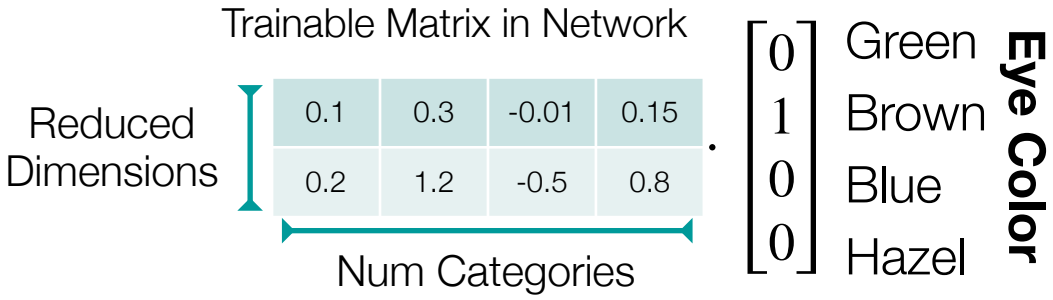
config = wandb.config )

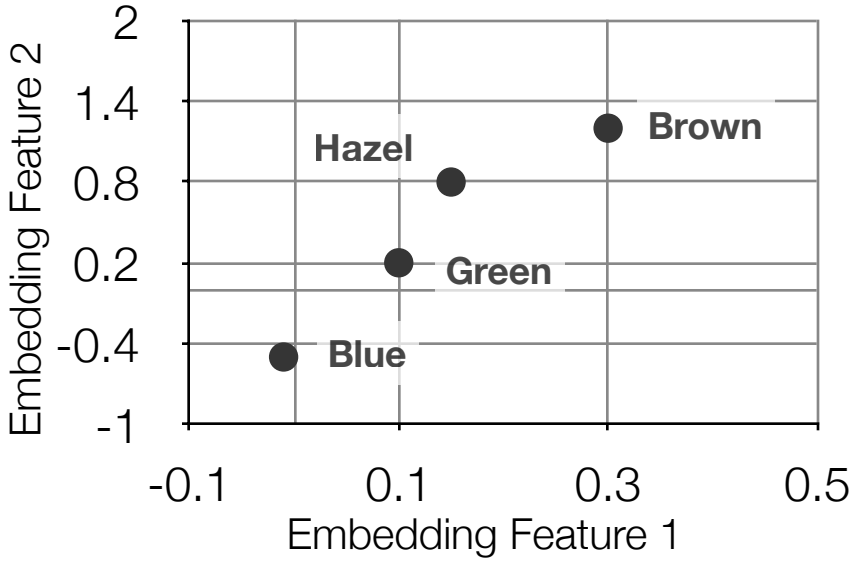
…

wandb.watch(model,
    log="all", log_freq=100)
```

Town Hall, Deep Networks (and Wide)







End of Session

- Next Time (or now if time):
  - Convolutional Neural Networks